

# Cloudflare Worker einrichten

Kontaktformular-Leads von zyntrix.co in die Telegram-Gruppe

Zyntrix · Branch `claude/telegram-bot-contact-form-d3i8pp` · Stand 7. September 2026

Voraussetzung: Telegram-Bot-Token und Gruppen-Chat-ID liegen bereits vor.

## 0 Was hier passiert

zyntrix.co liegt statisch auf GitHub Pages. Alles im Quelltext ist öffentlich lesbar – ein Bot-Token dürfte dort also nie stehen. Der Worker ist die Zwischenschicht, die das Token geheim hält:

```
Browser (zyntrix.co)
├── Web3Forms      ──> E-Mail (läuft schon)
└── Cloudflare Worker ──> Telegram-Gruppe (richten wir jetzt ein)
```

Öffentlich sichtbar ist am Ende nur die Worker-URL. Token und Chat-ID liegen als verschlüsselte Secrets bei Cloudflare.

**Kosten:** Der Cloudflare-Free-Plan erlaubt 100.000 Worker-Requests pro Tag. Für ein Kontaktformular ist das um Größenordnungen mehr als nötig – es fällt kein Cent an, eine Kreditkarte wird nicht verlangt.

**Zeitbedarf:** etwa 10 Minuten.

## 1 Zwei Wege - welchen nehmen?

| Weg                                                | Wann sinnvoll                                                                                                                               |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>A - Wrangler CLI</b><br>Terminal, Node.js nötig | Empfohlen. Die fertigen Dateien liegen bereits im Repo, du tippst vier Befehle. Spätere Änderungen sind ein einzelnes <code>deploy</code> . |
| <b>B - Dashboard</b><br>nur Browser                | Wenn du kein Node.js installieren willst oder magst. Alles per Klick, dafür Code einmal von Hand einfügen.                                  |

Beide führen zum identischen Ergebnis. Nimm **A**, wenn `node --version` in deinem Terminal eine Zahl ausgibt (Version 18 oder höher).

## 2 Cloudflare-Account

Falls noch keiner existiert: auf `dash.cloudflare.com/sign-up` registrieren und die Bestätigungsmail anklicken. Die Domain zyntrix.co muss **nicht** zu Cloudflare umgezogen werden – der Worker läuft eigenständig unter einer `workers.dev`-Adresse.

Beim ersten Worker fragt Cloudflare nach einer **workers.dev-Subdomain** (z. B. `zyntrix`). Die Wahl ist dauerhaft und Teil deiner späteren URL.

## 3 Weg A - Wrangler CLI

### 3.1 Ins Worker-Verzeichnis wechseln

```
# im Repo, Branch claude/telegram-bot-contact-form-d3i8pp
git pull
cd telegram-worker
```

Dort liegen `worker.js` (der Code), `wrangler.toml` (die Konfiguration) und eine Kurzfassung dieser Anleitung.

### 3.2 Anmelden

```
npx wrangler login
```

Öffnet den Browser, dort auf **Allow** klicken. Beim allerersten Aufruf fragt npx, ob das `wrangler` - Paket installiert werden darf – mit `y` bestätigen.

### 3.3 Secrets setzen

```
npx wrangler secret put TELEGRAM_BOT_TOKEN
npx wrangler secret put TELEGRAM_CHAT_ID
```

Nach jedem Befehl erscheint eine Eingabezeile. Dort den Wert einfügen und Enter drücken – die Eingabe wird bewusst nicht angezeigt.

| Secret             | Wert                                                                             |
|--------------------|----------------------------------------------------------------------------------|
| TELEGRAM_BOT_TOKEN | Das Token von BotFather, Form <code>123456789:AAF...</code>                      |
| TELEGRAM_CHAT_ID   | Die Gruppen-ID <b>inklusive Minuszeichen</b> , z. B. <code>-1001234567890</code> |

#### Häufigster Fehler

Das Minuszeichen der Chat-ID wird beim Kopieren abgeschnitten. Ohne Minus adressiert Telegram einen Privat-Chat statt der Gruppe und antwortet mit `chat not found`.

### 3.4 Deployen

```
npx wrangler deploy
```

Am Ende steht die URL, etwa:

```
https://zyntrix-telegram-lead.zyntrix.workers.dev
```

Diese URL notieren – sie wird in Schritt 5 gebraucht. Weiter bei **Schritt 5**.

---

## 4 Weg B - Dashboard, ohne CLI

### 4.1 Worker anlegen

1. Auf `dash.cloudflare.com` einloggen.
2. Links **Compute (Workers)** → Button **Create** → **Start with Hello World**.
3. Als Namen `zyntrix-telegram-lead` eintragen und auf **Deploy** klicken.

### 4.2 Code einsetzen

1. Nach dem Deploy auf **Edit code** (bzw. **Continue to project** → **Edit code**).
2. Im Repo `telegram-worker/worker.js` öffnen, den **kompletten** Inhalt kopieren.
3. Im Editor alles markieren (Strg / Cmd + A), einfügen, oben rechts **Deploy**.

### 4.3 Secrets und Variable setzen

Zurück im Worker: **Settings** → **Variables and Secrets** → **Add**. Drei Einträge:

| Name                            | Typ    | Wert                                                   |
|---------------------------------|--------|--------------------------------------------------------|
| <code>TELEGRAM_BOT_TOKEN</code> | Secret | Bot-Token                                              |
| <code>TELEGRAM_CHAT_ID</code>   | Secret | Gruppen-ID mit Minus                                   |
| <code>ALLOWED_ORIGINS</code>    | Text   | <code>https://zyntrix.co,https://www.zyntrix.co</code> |

Danach **Deploy** klicken, damit die Werte aktiv werden.

#### **ALLOWED\_ORIGINS nicht vergessen**

Beim CLI-Weg kommt der Wert automatisch aus `wrangler.toml`. Im Dashboard musst du ihn selbst anlegen – fehlt er, weist der Worker jede Anfrage mit `origin_not_allowed` ab. Kein Slash am Zeilenende, kein Leerzeichen nach dem Komma.

### 4.4 URL ablesen

Unter **Settings** → **Domains & Routes** steht die `workers.dev`-Adresse. Notieren.

---

## 5 URL in die Website eintragen

In `index.html` nach dieser Zeile suchen:

```
static TELEGRAM_ENDPOINT = '';
```

und die eigene URL einsetzen – **ohne** Slash am Ende:

```
static TELEGRAM_ENDPOINT = 'https://zyntrix-telegram-lead.zyntrix.workers.dev';
```

Dann committen und pushen:

```
git add index.html
git commit -m "Telegram-Worker-URL eintragen"
git push
```

GitHub Pages baut automatisch neu; nach ein bis zwei Minuten ist die Änderung live.

### Solange die Zeile leer bleibt

... ist der Telegram-Pfad schlicht inaktiv. Das Formular verschickt weiter E-Mails wie bisher. Du kannst den aktuellen Stand also gefahrlos deployen und die URL später nachtragen.

## 6 Testen

### 6.1 Worker direkt prüfen

```
curl -X POST https://DEINE-URL.workers.dev \
-H 'Content-Type: application/json' \
-H 'Origin: https://zyntrix.co' \
-d '{"name":"Testlauf","firma":"Testfirma GmbH",
    "email":"test@example.com","telefon":"+43 660 1234567",
    "zeitfresser":"Angebote schreiben"}'
```

Erwartet: `{"success":true}` und eine Nachricht in der Gruppe.

### 6.2 Echten Weg prüfen

Auf zyntrix.co das Formular einmal ausfüllen und absenden. Es müssen **beide** ankommen: die E-Mail wie gewohnt und die Nachricht in der Gruppe.

Der `Origin`-Header im curl-Test ist Pflicht – ohne ihn antwortet der Worker korrekterweise mit 403.

## 7 Wenn etwas nicht klappt

Live-Logs des Workers ansehen:

```
npx wrangler tail
```

Im Dashboard alternativ unter **Logs** → **Live**. Dann in einem zweiten Fenster den curl-Test wiederholen.

| Antwort                         | Ursache und Lösung                                                                                                                                                        |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>origin_not_allowed</code> | <code>ALLOWED_ORIGINS</code> passt nicht zur aufrufenden Domain. Auf Tippfehler, Slash am Ende und <code>www</code> -Variante prüfen.                                     |
| <code>not_configured</code>     | Ein Secret fehlt. Mit <code>npx wrangler secret list</code> prüfen, ob beide gesetzt sind.                                                                                |
| <code>telegram_failed</code>    | Telegram lehnt ab. Fast immer: Chat-ID ohne Minus, Bot nicht in der Gruppe, oder Bot dort nicht Administrator. Der genaue Grund steht im <code>wrangler tail</code> -Log. |
| <code>invalid_payload</code>    | Der Request-Body ist kein gültiges JSON – meist ein Zeilenumbruch im curl-Aufruf.                                                                                         |
| <code>empty_lead</code>         | Weder Name noch E-Mail übergeben.                                                                                                                                         |
| E-Mail kommt, Telegram nicht    | Die URL in <code>index.html</code> fehlt oder ist falsch. In der Browser-Konsole (F12) nach einem Fehler zu <code>workers.dev</code> sehen.                               |

## 8 Gut zu wissen

### Was der Worker abwehrt

- **Origin-Prüfung** – nur die gelisteten Domains werden angenommen, fremde Websites bekommen 403.
- **Honeypot** – ein ausgefülltes `botcheck`-Feld wird stillschweigend verworfen.
- **Escaping und Längenbegrenzung** – Felder werden auf 1500 Zeichen gekürzt, HTML wird neutralisiert. Niemand kann über das Formular Markup in eure Gruppe schleusen.

#### Grenze der Origin-Prüfung

Sie stoppt Aufrufe aus fremden Websites, aber nicht jemanden mit `curl`, der den Origin-Header selbst setzt – dieser Header ist frei wählbar. Der Worker kann darum theoretisch als Spam-Kanal in eure Gruppe missbraucht werden, wenn jemand die URL findet. Falls das je passiert: in Cloudflare unter

#### Security

eine Rate-Limiting-Regel auf die Worker-Route legen (z. B. 5 Requests pro Minute pro IP) oder Turnstile vorschalten.

### Telegram-Gruppe mit Topics

Nutzt die Gruppe Themen/Topics und sollen die Leads in ein bestimmtes Topic, in `wrangler.toml` ergänzen:

```
TELEGRAM_THREAD_ID = "12"
```

Die Zahl steht in der Topic-URL. Danach erneut `npx wrangler deploy`. Im Dashboard entsprechend als Text-Variable anlegen.

## Bot-Token gewechselt

```
npx wrangler secret put TELEGRAM_BOT_TOKEN
```

Überschreibt den alten Wert. Kein erneutes Deploy nötig, kein Eingriff in die Website.

## Weitere Formularfelder

Neue Felder im Formular landen automatisch in der Telegram-Nachricht – der Worker gibt alles aus, was er bekommt. Für eine schöne deutsche Beschriftung statt des rohen Feldnamens die Liste `FIELD_LABELS` oben in `worker.js` ergänzen.